

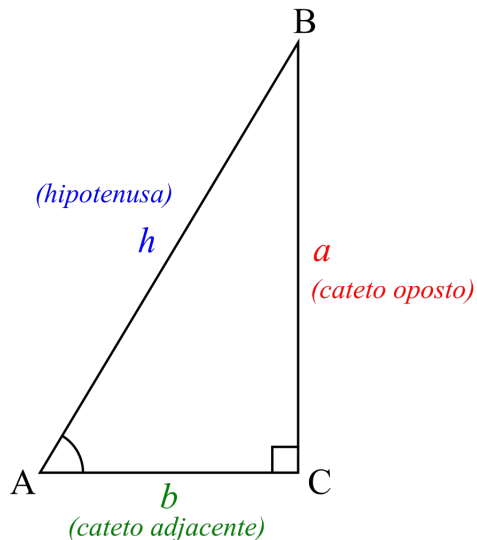
Basic math with Python

Universidade Federal de Pernambuco

Adrien Durand-Petiteville

`adrien.durandpetiteville@ufpe.br`

Trigonometric functions



$$\operatorname{sen} \alpha = \frac{\text{cateto oposto}}{\text{hipotenusa}} = \frac{a}{h}$$

$$\cos \alpha = \frac{\text{cateto adjacente}}{\text{hipotenusa}} = \frac{b}{h}$$

$$\tan \alpha = \frac{\text{cateto oposto}}{\text{cateto adjacente}} = \frac{a}{b}$$

```
import math
```

Trigonometric functions

- **`math.cos(x)`**
Return the cosine of x radians.
- **`math.sin(x)`**
Return the sine of x radians.
- **`math.tan(x)`**
Return the tangent of x radians.
- **`math.dist(p, q)`**
Return the Euclidean distance between two points p and q, each given as a sequence (or iterable) of coordinates. The two points must have the same dimension.

Trigonometric functions - 2

- **`math.acos(x)`**

Return the arc cosine of x , in radians. The result is between 0 and π .

- **`math.asin(x)`**

Return the arc sine of x , in radians. The result is between $-\pi/2$ and $\pi/2$.

- **`math.atan(x)`**

Return the arc tangent of x , in radians. The result is between $-\pi/2$ and $\pi/2$.

- **`math.atan2(y, x)`**

Return $\text{atan}(y / x)$, in radians. The result is between $-\pi$ and π . The vector in the plane from the origin to point (x, y) makes this angle with the positive X axis. The point of `atan2()` is that the signs of both inputs are known to it, so it can compute the correct quadrant for the angle. For example, `atan(1)` and `atan2(1, 1)` are both $\pi/4$, but `atan2(-1, -1)` is $-3\pi/4$.

Angular conversion

- **`math.degrees(x)`**
Convert angle x from radians to degrees.
- **`math.radians(x)`**
Convert angle x from degrees to radians.

- **math.pi**

The mathematical constant $\pi = 3.141592\dots$

- **math.e**

The mathematical constant $e = 2.718281\dots$

- **math.inf**

A floating-point positive infinity. (For negative infinity, use `-math.inf`.)
Equivalent to the output of `float('inf')`.

- **math.nan**

A floating-point “not a number” (NaN) value.
Equivalent to the output of `float('nan')`

Representation functions

- **`math.isfinite(x)`**
Return True if x is neither an infinity nor a NaN, and False otherwise.
Note that 0.0 is considered finite.
- **`math.isinf(x)`**
Return True if x is a positive or negative infinity, and False otherwise.
- **`math.isnan(x)`**
Return True if x is a NaN (not a number), and False otherwise.

Representation functions - 2

- **`math.ceil(x)`**
Return the ceiling of x , the smallest integer greater than or equal to x .
- **`math.floor(x)`**
Return the floor of x , the largest integer less than or equal to x .
- **`math.trunc(x)`**
Return x with the fractional part removed, leaving the integer part.
- **`math.fabs(x)`**
Return the absolute value of x .

Power and logarithmic functions

- **`math.pow(x, y)`**
Return x raised to the power y .
- **`math.sqrt(x)`**
Return the square root of x .
- **`math.exp(x)`**
Return e raised to the power x , where $e = 2.718281\dots$ is the base of natural logarithms.
- **`math.log(x[, base])`**
With one argument, return the natural logarithm of x (to base e).
With two arguments, return the logarithm of x to the given base.
- **`math.log2(x)`**
Return the base-2 logarithm of x . This is usually more accurate than `log(x, 2)`.
- **`math.log10(x)`**
Return the base-10 logarithm of x . This is usually more accurate than `log(x, 10)`.

```
import numpy as np
```

Creating matrices

You can pass Python lists of lists to create a 2-D array (or “matrix”) to represent them in NumPy.

```
>>> data = np.array([[1, 2], [3, 4], [5, 6]])  
>>> data  
array([[1, 2],  
       [3, 4],  
       [5, 6]])
```

`np.array([[1,2],[3,4],[5,6]])`



1	2
3	4
5	6

Indexing and slicing

Indexing and slicing operations are useful when you're manipulating matrices:

```
>>> data[0, 1]  
2  
>>> data[1:3]  
array([[3, 4],  
       [5, 6]])  
>>> data[0:2, 0]  
array([1, 3])
```

	data	
	0	1
0	1	2
1	3	4
2	5	6

	data[0,1]	
	0	1
0	1	2
1	3	4
2	5	6

	data[1:3]	
	0	1
0	1	2
1	3	4
2	5	6

	data[0:2,0]	
	0	1
0	1	2
1	3	4
2	5	6

Aggregation

You can aggregate matrices the same way you aggregated vectors:

```
>>> data.max()
6
>>> data.min()
1
>>> data.sum()
21
```

data	
1	2
3	4
5	6

`.max()` = 6

data	
1	2
3	4
5	6

`.min()` = 1

data	
1	2
3	4
5	6

`.sum()` = 21

Addition

Once you've created your matrices, you can add and multiply them using arithmetic operators if you have two matrices that are the same size.

```
>>> data = np.array([[1, 2], [3, 4]])  
>>> ones = np.array([[1, 1], [1, 1]])  
>>> data + ones  
array([[2, 3],  
       [4, 5]])
```

$$\begin{array}{c} \text{data} \\ \text{data} + \text{ones} \end{array} = \begin{array}{|c|c|} \hline 1 & 2 \\ \hline 3 & 4 \\ \hline \end{array} + \begin{array}{|c|c|} \hline 1 & 1 \\ \hline 1 & 1 \\ \hline \end{array} = \begin{array}{|c|c|} \hline 2 & 3 \\ \hline 4 & 5 \\ \hline \end{array}$$

Ones, Zeros and Random

You can also use `ones()`, `zeros()`, and `random()` to create a 2D array if you give them a tuple describing the dimensions of the matrix:

```
>>> np.ones((3, 2))
array([[1., 1.],
       [1., 1.],
       [1., 1.]])
>>> np.zeros((3, 2))
array([[0., 0.],
       [0., 0.],
       [0., 0.]])
>>> rng.random((3, 2))
array([[0.01652764, 0.81327024],
       [0.91275558, 0.60663578],
       [0.72949656, 0.54362499]]) # may vary
```

Matrix multiplication

- **numpy.matmul(x1, x2)**
 - x1, x2: array
Input arrays, scalars not allowed.
 - y: array
The matrix product of the inputs

For 2-D arrays it is the matrix product:

```
>>> a = np.array([[1, 0],  
...               [0, 1]])  
>>> b = np.array([[4, 1],  
...               [2, 2]])  
>>> np.matmul(a, b)  
array([[4, 1],  
       [2, 2]])
```

Matrix inverse

- **numpy.linalg.inv(a)**

Given a **square** matrix, compute the inverse of the matrix.

- a: array
Matrix to be inverted.
- ainv: array
Inverse of the matrix a

```
>>> from numpy.linalg import inv
>>> a = np.array([[1., 2.], [3., 4.]])
>>> ainv = inv(a)
>>> np.allclose(a @ ainv, np.eye(2))
True
>>> np.allclose(ainv @ a, np.eye(2))
True
```